

AI-Powered Local Coding Assistant

Capstone Project – Problem Statement and Solution Guide

1. Problem Statement

Develop a web-based AI Coding Assistant that runs entirely in a local development environment and provides an interactive coding experience similar in concept to modern AI-powered IDE assistants such as Google's Antigravity. The application should allow a developer to interact with a local project through a browser. The developer should be able to provide coding requests in natural language, inspect the existing codebase, ask questions about the code, generate or modify source code, identify potential problems, and execute selected operations through an AI-driven workflow.

The solution must use Flask as the web application framework, LangChain for LLM interaction and tool integration, and LangGraph for orchestrating multiple specialized AI agents. The objective is not to build a complete IDE, but to demonstrate how a multi-agent coding assistant can understand a software project, reason about developer requests, invoke appropriate tools, and coordinate multiple agents to accomplish coding tasks.

2. Expected User Experience

1. Select or configure a local project directory.
2. Browse the project's files.
3. Select a file and view its contents.
4. Enter a natural-language coding request.
5. Allow the AI system to analyze the request and existing code.
6. Automatically determine which agents need to participate.
7. Inspect relevant files and dependencies.
8. Generate or modify code.
9. Review the proposed changes.
10. Execute appropriate tests or validation commands.
11. Receive an explanation of the changes and validation results.

Example request:

"Add input validation to the /users API and create unit tests for the validation logic."

3. Core Functional Requirements

3.1 Project Workspace

- Allow the user to configure a local project directory.

- List files and directories.
- Read source files.
- Search for files and text/code patterns.
- Identify the project's programming language and commonly used frameworks/libraries.
- Ignore directories such as .git, node_modules, __pycache__, .venv, target, build and dist.
- Restrict file operations to the configured project directory.

3.2 Natural Language Coding Requests

- Support requests such as explaining code, locating implementation, adding features, fixing bugs, generating tests, refactoring code, identifying security issues and diagnosing API/runtime errors.
- The system should classify the request and determine the appropriate workflow.

4. Required Multi-Agent Architecture

Students should not implement the entire application using one general-purpose agent. The solution should use multiple specialized agents coordinated using LangGraph.

Agent 1: Supervisor / Orchestrator Agent

- Understand the user's request.
- Determine the type of task.
- Identify which specialized agents are required.
- Decide the execution sequence.
- Maintain the overall workflow.
- Handle failures and route the workflow appropriately.

Agent 2: Codebase Explorer Agent

- Explore directories and identify relevant files.
- Search source code and locate classes/functions.
- Identify relationships between files.
- Determine where a requested feature is implemented.
- Provide relevant context to downstream agents.

Agent 3: Code Analysis Agent

- Understand existing implementation.
- Identify dependencies and trace method calls.
- Analyze control flow.
- Identify potential problems.
- Explain existing code and identify modification points.

Agent 4: Code Generation / Modification Agent

- Generate new code and modify existing code.
- Refactor code and add methods/classes/functions.
- Generate configuration changes when required.
- Produce a structured change proposal.

Agent 5: Test Generation Agent

- Identify the appropriate testing framework.
- Analyze existing tests and follow project conventions.
- Generate unit tests, negative scenarios and boundary-condition tests.

Agent 6: Test Execution Agent

- Determine the appropriate test command.
- Execute tests.
- Capture stdout/stderr.
- Detect failures and return structured test results.

Agent 7: Debugging Agent

- Analyze stack traces, compiler errors, test failures and runtime errors.
- Inspect relevant source code.
- Identify probable causes and propose fixes.

Agent 8: Code Review Agent

- Review correctness, maintainability and code quality.
- Identify potential bugs, security issues, duplication, unnecessary changes and error-handling problems.

Agent 9: Documentation Agent

- Generate function/class documentation, API documentation, README updates, change summaries and developer explanations.
- This may be implemented as an advanced extension.

5. Suggested LangGraph State

A shared state can contain fields similar to the following:

- user_request: str
- project_path: str
- task_type: str
- relevant_files: list[str]
- file_contents: dict[str, str]
- analysis: str
- proposed_changes: list[dict]
- review_result: str
- test_command: str
- test_output: str
- test_passed: bool
- error_details: str
- final_response: str

6. Suggested LangGraph Workflow

A representative workflow is Supervisor → Codebase Explorer → Code Analysis → Code Generation → Code Review → Human Approval → Test Generation → Test Execution. If tests fail, the workflow should route to the Debugging Agent and then iterate through fix, review and test execution.

7. Suggested Architecture Diagram

```
flowchart TB
    USER["Developer"]

    subgraph WEB["Web Application"]
        UI["Browser UI"]
        FLASK["Flask Application"]
    end

    subgraph ORCHESTRATION["AI Orchestration Layer"]
        GRAPH["LangGraph Workflow"]
        SUP["Supervisor Agent"]
        EXP["Codebase Explorer Agent"]
        ANALYZE["Code Analysis Agent"]
        GENERATE["Code Generation Agent"]
        DEBUG["Debugging Agent"]
        TESTGEN["Test Generation Agent"]
        TESTEXEC["Test Execution Agent"]
        REVIEW["Code Review Agent"]
        DOC["Documentation Agent"]
    end

    subgraph LLM["LLM Layer"]
        MODEL["LLM"]
        EMBED["Embedding Model"]
    end

    subgraph TOOLS["Coding Tools"]
        FILES["File System Tools"]
        SEARCH["Code Search Tool"]
        SHELL["Shell / Command Execution Tool"]
        GIT["Git Tools"]
    end

    subgraph PROJECT["Local Project"]
        SOURCE["Source Code"]
        TESTS["Tests"]
        CONFIG["Configuration Files"]
        REPO["Git Repository"]
    end

    USER --> UI
    UI --> FLASK
    FLASK --> GRAPH
    GRAPH --> SUP
    SUP --> EXP
    SUP --> ANALYZE
    SUP --> GENERATE
    SUP --> DEBUG
    SUP --> TESTGEN
    SUP --> TESTEXEC
```

```

SUP --> REVIEW
SUP --> DOC
EXP --> FILES
EXP --> SEARCH
ANALYZE --> FILES
ANALYZE --> SEARCH
GENERATE --> FILES
GENERATE --> MODEL
DEBUG --> FILES
DEBUG --> SHELL
DEBUG --> MODEL
TESTGEN --> FILES
TESTGEN --> MODEL
TESTEXEC --> SHELL
REVIEW --> FILES
REVIEW --> MODEL
DOC --> MODEL
DOC --> FILES
FILES --> PROJECT
SEARCH --> PROJECT
SHELL --> PROJECT
GIT --> REPO
MODEL --> LLM
MBED --> LLM

```

8. Custom LangChain Tools

- File tools: `list_directory()`, `read_file()`, `write_file()`, `create_file()`, `delete_file()`.
- Search tools: `search_files()`, `search_code()`, `find_definition()`.
- Execution tools: `execute_command()`, `run_tests()`.
- Git tools: `git_status()`, `git_diff()`, `git_log()`.

9. Human-in-the-Loop

Before modifying a file, the system should present a proposed change to the user and pause the LangGraph workflow. The user should be able to approve or reject the change. Approval resumes the graph and applies the modification. This demonstrates LangGraph interrupts, persistence and resume.

10. Example End-to-End Scenario

User: Add a REST endpoint to retrieve an order by ID.

Supervisor: Classifies the request as `FEATURE_IMPLEMENTATION` and routes it.

Explorer: Finds `OrderController`, `OrderService`, `OrderRepository` and `Order`.

Analysis Agent: Determines the relevant call flow and modification points.

Code Generation Agent: Proposes changes to the appropriate controller/service files.

Code Review Agent: Reviews the proposed implementation.

Human Approval: The UI displays the proposed modifications for approval.

Test Generation Agent: Generates controller/service tests.

Test Execution Agent: Runs the appropriate test command.

Debugging Agent: If tests fail, analyzes the failure and routes the workflow back for correction.

11. Technology Requirements

Technology	Responsibility
Python	Primary programming language
Flask	Web application/backend
LangChain	LLM integration, prompts and tools
LangGraph	Multi-agent orchestration and workflow
HTML/CSS/JavaScript	Browser interface
LLM	Code understanding and generation
Git	Optional project/version-control integration

12. Key LangChain Concepts Expected

- Chat models
- Prompt templates
- Structured output
- Tool calling
- Custom tools
- Agents
- Message handling
- Context management
- LLM-based code analysis

13. Key LangGraph Concepts Expected

- StateGraph
- Shared state
- Nodes
- Edges
- Conditional edges
- Agent orchestration
- Loops
- Checkpoints/persistence
- Human-in-the-loop
- Interrupt/resume
- Error handling
- Workflow termination

14. Security Requirements

12. Restrict file access to the configured project directory.
13. Prevent path traversal.
14. Avoid unrestricted shell execution.
15. Validate commands before execution.
16. Use an execution allowlist where practical.
17. Display proposed file modifications before applying them.
18. Require human approval for potentially destructive operations.
19. Never expose API keys to the browser.
20. Keep secrets in environment variables.
21. Maintain an operation log.

15. Minimum Viable Product

- Project selection/configuration
- File explorer
- Natural-language coding request
- Supervisor Agent
- Codebase Explorer Agent
- Code Analysis Agent
- Code Generation Agent
- Human approval before applying changes
- Test Generation Agent
- Test Execution Agent
- Final result and explanation

16. Advanced Extensions

- Multi-file reasoning
- Git integration and diff/commit support
- Codebase RAG using embeddings and a vector store
- Persistent project conversations
- Automatic bug fixing with iterative test-and-repair loops
- Support for multiple programming languages
- Streaming agent results to the Flask UI

17. Final Project Objective

The completed system should demonstrate how a local AI coding assistant can be constructed as a multi-agent system rather than as a conventional chatbot. The central learning outcome is to build a real multi-agent software-engineering workflow using LangGraph, with LangChain providing the LLM and tooling layer and Flask providing the local web-based user interface.